Vel Tech Group of Institutions

Name: Gontu Yaswanth
Email: vtu33434@veltech.edu.in
Roll no: Vtu33434
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 4_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

A financial analytics company is developing a tool to analyze stock price trends. The tool needs to identify the Next Greater Price for each stock price in a given list of daily prices. For a given stock price, the Next Greater Price is defined as the first price appearing after it in the list that is greater. If no such price exists, the result should be -1.

Note: The Next greater Element for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exist, consider the next greater element as -1.

Examples:

Input Format

The first line of input consists of an integer n, representing the size of the array.

The second line consists of n space-separated stack of integers.

Output Format

The output prints "Next Greater Elements: " followed by an array of integers, representing the next greater elements in the stack.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
4 5 2 25
Output: Next Greater Elements: 5 25 25 -1

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[20], result[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        result[i] = -1; // default value
    }

    // Logic (simple brute force)
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[j] > arr[i]) {
                result[i] = arr[j];
                break;
            }
        }
```

```
    }

    printf("Next Greater Elements: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", result[i]);
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement:

Riya is working on a text-cleaning utility where certain unwanted patterns must be removed from user input. She uses a stack data structure to process the string character by character.

Given a string containing alphabets, digits, and special characters, write a program to ensure that the task is done by removing every digit along with the closest non-digit character immediately to its left. The remaining characters should be preserved in their original order and printed as the final cleaned string.

Example:

Input:

s = "cb34"

Output:

""

Explanation:

Start with an empty stack.

Push c stack: ['c']

Push b stack: ['c','b']

Encounter 3 pop 'b' stack: ['c']

Encounter 4 pop 'c' stack: []

Result: empty string

Input Format

The input consists of a string s without spaces, consisting of letters, digits, and special characters.

Output Format

The output prints the resulting string after performing the required removals.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abc

Output: abc

Answer

```
# You are using Python
s = input().strip()

stack = []

for ch in s:
    if ch.isdigit():
        if stack:
            stack.pop()   # remove closest non-digit
    else:
        stack.append(ch)

print("".join(stack))
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Kavya is developing a real-time analytics system that processes a

continuous stream of numerical data. To efficiently track trends, she uses a Queue data structure to maintain incoming values and compute statistics incrementally.

Given a sequence of integers arriving one by one, write a program to ensure that the task is done by calculating and printing the moving average of all elements received so far after each new input. The average should be displayed up to two decimal places.

Example

Input

3

1 2 3

Output

1.00 1.50 2.00

Explanation

The moving average of the first element is 1.00.

The next elements are 1 and 2, the moving average is (1 + 2) / 2 = 1.50.

The next elements are 1, 2, and 3, the moving average is (1 + 2 + 3) / 3 = 2.00.

Input Format

The first line contains an integer n, representing the number of elements in the data stream.

The second line contains n space-separated integers representing the incoming values.

Output Format

The output prints n floating-point numbers, each representing the moving average after processing the corresponding element.

Each value should be printed with two decimal places, separated by spaces.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3

Output: 1.00 1.50 2.00

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];
    double sum = 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for (int i = 0; i < n; i++) {
        sum += arr[i];
        printf("%.2f ", sum / (i + 1));
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

In a small business, Jane is tasked with organizing customer feedback scores for a new product. Each customer has rated the product, and Jane needs to arrange these ratings in a specific order to analyze the feedback more effectively.

Specifically, she wants to rearrange the ratings such that the most frequent ratings appear first. If two ratings have the same frequency, the smaller rating should come first. To accomplish this, assist Jane with a program to process a queue of customer ratings and rearrange them based on their frequencies.

Input Format

The first line contains an integer n, which represents the number of customer ratings.

The second line contains n integers, each representing a customer rating.

Output Format

The output displays the rearranged ratings in the order of their frequency (most frequent first).

If frequencies are the same, the smaller rating should come first.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7
2 3 2 3 2 3 2
Output: 2 2 2 2 3 3 3

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];
    int freq[100] = {0}; // for counting

    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
        freq[arr[i]]++;
    }

    // sort based on frequency and value
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (freq[arr[i]] < freq[arr[j]] ||
                (freq[arr[i]] == freq[arr[j]] && arr[i] > arr[j])) {

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    // print result
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***